

SHOPFORGE

Frontend Design System & Dev Plan

Next.js 14 (App Router) · Tailwind CSS · shadcn/ui · Zustand · TanStack Query
6 Sprints · 12 Weeks · Production-Ready

1. Design Philosophy

ShopForge targets urban Indian shoppers aged 18–35. The aesthetic must feel premium and trustworthy — not a Shopify clone — while staying warm and approachable. Three principles guide every decision:

Principle	What it means in practice
Warm Precision	Saffron accents on a clean navy/white base — premium without coldness. Every interactive element has a warm glow state.
Mobile-first Commerce	Designed at 390px first. Bottom nav, thumb-friendly tap targets (min 44px), sticky cart CTA always visible.
Speed as UX	Next.js SSR for product pages (SEO), client-side cart/filters (instant feel). Images via next/image with blur placeholder.

2. Design Tokens — The Token System

2.1 Colour Palette

Six named colours. The saffron (#F59E0B) is the signature — it appears only on CTAs, active states, and key highlights. Everything else uses navy and slate.

Swatch	Name / Hex	Usage
	Ink Navy — Primary #0F172A	Main background, headings, navbar bg, footer
	Royal Blue — Secondary #1E40AF	Links, badge backgrounds, info states
	Saffron — Accent ★ #F59E0B	All CTAs, hover states, active nav items, price highlights, cart badge
	Off-White — Surface #FAFAFA	Page backgrounds, card surfaces
	Slate — Muted #64748B	Body text, labels, placeholder text, borders
	Alert Red — Danger #EF4444	Out of stock, errors, destructive actions only

2.2 Typography Scale

Role	Font Family	Weight	Size	Use
Display / Hero	Plus Jakarta Sans	800 ExtraBold	48–72px	Hero headline, product name H1
Heading 1	Plus Jakarta Sans	700 Bold	32px	Page titles, section headers
Heading 2	Plus Jakarta Sans	600 SemiBold	24px	Card titles, section sub-headers
Heading 3	Plus Jakarta Sans	600 SemiBold	18px	Sidebar labels, accordion headers
Body	Inter	400 Regular	16px	All body copy, descriptions
Body Small	Inter	400 Regular	14px	Meta info, timestamps, helper text
Label / Caption	Inter	500 Medium	12px	Tags, badges, form labels
Price	Plus Jakarta Sans	700 Bold	20–28px	Product prices — always saffron
Code	JetBrains Mono	400	14px	Order numbers, tracking codes, SKUs

2.3 Spacing & Radius

Token	Value	Usage
space-1	4px	Icon padding, tight gaps
space-2	8px	Badge padding, inline gaps
space-3	12px	Input padding vertical
space-4	16px	Card padding, section gaps
space-6	24px	Component gap (header, sidebar)
space-8	32px	Section padding
space-12	48px	Page section top/bottom
space-20	80px	Hero vertical padding
radius-sm	6px	Inputs, chips, badges
radius-md	12px	Cards, dropdowns, modals
radius-lg	20px	Image thumbnails, avatars
radius-xl	28px	Hero image, category cards
radius-full	9999px	Pill buttons, toggle handles

2.4 Shadows & Elevation

Token	CSS Value	Use
shadow-card	0 1px 3px rgba(0,0,0,.08), 0 1px 2px rgba(0,0,0,.06)	Product cards at rest
shadow-card-hover	0 10px 25px rgba(0,0,0,.12), 0 4px 6px rgba(0,0,0,.05)	Product cards on hover (translate-y-1)
shadow-modal	0 20px 60px rgba(0,0,0,.3)	Drawers, modals, cart sidebar
shadow-saffron	0 0 0 3px rgba(245,158,11,.35)	Focus ring on all interactive elements
shadow-nav	0 2px 8px rgba(15,23,42,.15)	Sticky navbar drop shadow

2.5 Motion

Animation	Duration / Easing	Applied to
Fade in up	300ms ease-out + translateY(8px → 0)	Page sections on scroll (Intersection Observer)
Cart slide-in	280ms cubic-bezier(.32,0,.67,0)	Cart drawer from right

Animation	Duration / Easing	Applied to
Saffron pulse	600ms ease-in-out, 1 iteration	Add-to-cart success feedback on cart icon
Skeleton shimmer	1.5s linear infinite	Loading states for product cards
Hover lift	200ms ease + translateY(-2px)	Product cards on hover
Image zoom	400ms ease	Product thumbnail hover — scale(1.04)

3. Component Library

3.1 Tailwind Config (tailwind.config.ts)

```
import type { Config } from 'tailwindcss'

export default {
  darkMode: 'class',
  content: ['./src/**/*.ts', './src/**/*.tsx'],
  theme: {
    extend: {
      colors: {
        navy: { DEFAULT: '#0F172A', light: '#1E293B' },
        saffron: { DEFAULT: '#F59E0B', light: '#FCD34D', dark: '#B45309' },
        royal: { DEFAULT: '#1E40AF', light: '#3B82F6' },
        slate: { DEFAULT: '#64748B' },
        surface: { DEFAULT: '#FAFAFA', card: '#FFFFFF' },
        danger: { DEFAULT: '#EF4444' },
      },
      fontFamily: {
        display: ['Plus Jakarta Sans', 'sans-serif'],
        body: ['Inter', 'sans-serif'],
        mono: ['JetBrains Mono', 'monospace'],
      },
      boxShadow: {
        card: '0 1px 3px rgba(0,0,0,.08), 0 1px 2px rgba(0,0,0,.06)',
        'card-hover': '0 10px 25px rgba(0,0,0,.12), 0 4px 6px rgba(0,0,0,.05)',
        modal: '0 20px 60px rgba(0,0,0,.3)',
        saffron: '0 0 0 3px rgba(245,158,11,.35)',
      },
      borderRadius: {
        xl: '20px', '2xl': '28px',
      },
      animation: {
        'fade-up': 'fadeUp 300ms ease-out',
        'pulse-saffron': 'pulseSaffron 600ms ease-in-out 1',
        shimmer: 'shimmer 1.5s linear infinite',
      },
      keyframes: {
        fadeUp: { from: { opacity: '0', transform: 'translateY(8px)' }, to: { opacity: '1', transform: 'translateY(0)' } },
        pulseSaffron: { '0%,100%': { boxShadow: '0 0 0 0 rgba(245,158,11,0)' }, '50%': { boxShadow: '0 0 0 8px rgba(245,158,11,.4)' } },
        shimmer: { from: { backgroundPosition: '-200% 0' }, to: { backgroundPosition: '200% 0' } },
      },
    },
  },
  plugins: [require('@tailwindcss/forms'), require('@tailwindcss/line-clamp')],
} satisfies Config
```

3.2 Core Component Patterns

Component	File	Key Classes / Notes
Button – Primary	components/ui/Button.tsx	bg-saffron text-navy font-display font-700 rounded-full px-6 py-3 hover:bg-saffron-dark focus:shadow-saffron transition-all
Button – Secondary	components/ui/Button.tsx	border-2 border-navy text-navy hover:bg-navy hover:text-saffron rounded-full
Button – Ghost	components/ui/Button.tsx	text-slate hover:text-navy hover:bg-navy/5
ProductCard	components/catalog/ProductCard.tsx	rounded-2xl shadow-card hover:shadow-card-hover hover:-translate-y-1 transition-all duration-200
PriceTag	components/catalog/PriceTag.tsx	font-display font-700 text-saffron; strikethrough original in slate
Badge	components/ui/Badge.tsx	rounded-full px-2 py-0.5 text-[12px] font-500; variants: NEW(navy), SALE(saffron), OUT(red/10 text-red)
RatingStars	components/ui/RatingStars.tsx	Filled stars in saffron; empty in slate/20
CartDrawer	components/cart/CartDrawer.tsx	Fixed right panel, w-[420px] mobile=full, shadow-modal, slide-in animation
SkeletonCard	components/ui/Skeleton.tsx	bg-gradient shimmer animation on bg-slate/10
FilterSidebar	components/catalog/FilterSidebar.tsx	sticky top-[72px] max-h-[calc(100vh-80px)] overflow-y-auto
SearchModal	components/search/SearchModal.tsx	Full-screen overlay on mobile; centered dialog on desktop; cmd+K trigger
BottomNav	components/layout/BottomNav.tsx	md:hidden fixed bottom-0 w-full; active tab gets saffron icon + label

3.3 Product Card — Full Spec

```
// components/catalog/ProductCard.tsx
interface ProductCardProps {
  product: {
    id: string; slug: string; name: string; primaryImage: string;
    basePrice: number; salePrice?: number; avgRating: number;
    reviewCount: number; isNew?: boolean; badge?: 'SALE' | 'NEW' | 'HOT';
    brand: string; inStock: boolean;
  }
}

// Structure:
// <article class='group relative rounded-2xl bg-white shadow-card hover:shadow-card-hover
//           hover:-translate-y-1 transition-all duration-200 overflow-hidden'>
//   <div class='relative aspect-[4/3] overflow-hidden'>
//     <Image fill sizes='...' class='object-cover group-hover:scale-[1.04] transition-
// transform duration-400' />
//     <WishlistButton class='absolute top-3 right-3 opacity-0 group-hover:opacity-100' />
```

```
//      <Badge class='absolute top-3 left-3' />    // NEW / SALE / HOT
//      {!inStock && <OutOfStockOverlay/>}
//    </div>
//    <div class='p-4 space-y-1'>
//      <p class='text-xs text-slate font-500 uppercase tracking-wider'>{brand}</p>
//      <h3 class='font-display font-600 text-navy text-[15px] line-clamp-2'>{name}</h3>
//      <RatingStars rating={avgRating} count={reviewCount}/>
//      <PriceTag base={basePrice} sale={salePrice}/>
//      <AddToCartButton variantId={...}/>    // shows on hover on desktop, always visible
mobile
//    </div>
//  </article>
```

4. Page-by-Page Design Spec

4.1 Homepage (/)

Section	Layout	Key Design Notes
Navbar	Sticky; navy bg; logo left, search centre, icons right	Search triggers full-screen overlay. Cart icon with saffron badge count.
Hero Banner	Full-width, 90vh on desktop	Bold display headline in white on navy-to-royal gradient. CTA button: saffron pill. Rotating product image right side. Mobile: stacked, headline first.
Category Grid	4-col desktop, 2-col mobile; rounded-2xl image cards	Category image fills card; label overlaid on bottom navy gradient. Hover: scale(1.02).
Flash Sale Strip	Full-width saffron background section	Countdown timer. Horizontal scroll of product cards. 'Ends in' text in navy bold.
Featured Products	3-col desktop, 2-col mobile grid	Section title with saffron underline accent. Standard ProductCards.
Brand Logos Row	Horizontal scroll strip; grayscale on rest, colour on hover	Shows trusted seller brands.
Trust Badges	4-col icon + text row	Free Shipping / Easy Returns / Secure Pay / 24/7 Support. Icons in saffron.
Footer	4-col link grid + bottom strip	Navy background. Saffron brand name. Social icons. Copyright.

4.2 Product Listing Page (/c/[slug])

Element	Spec
Layout	Left sidebar (240px) + main grid; sidebar collapses to bottom sheet on mobile
Filter Sidebar	Accordion sections: Category, Brand, Price Range (dual slider), Rating, Availability. Applied filters shown as removable chips above grid.
Sort Dropdown	Tailwind select; options: Relevance, Price Low → High, Price High → Low,

Element	Spec
	Newest, Best Rated, Best Selling
Product Grid	4-col desktop / 2-col mobile. Skeleton cards on load. Infinite scroll OR pagination toggle.
Active Filters Bar	Horizontally scrollable chip row. Each chip: slate bg, X to remove. 'Clear all' in saffron.
Result Count	'Showing 1–24 of 318 results' in slate/12. Updates on filter change.
Empty State	Illustration + 'No products found' + suggestions to widen filters.

4.3 Product Detail Page (/p/[slug])

Element	Spec
Image Gallery	Left: thumbnail strip (vertical desktop / horizontal mobile). Right: main image with pinch-zoom on mobile. Lightbox on click desktop.
Product Info Block	Brand in slate caps. Name in display/700/32px. Rating row. Price with sale badge.
Variant Selector	Color: circular swatches with saffron ring on selected. Size: pill buttons, disabled+strikethrough if OOS.
Stock Status	In Stock: green dot + text. Low Stock: saffron warning. OOS: red badge.
Add to Cart / Buy Now	2 full-width buttons stacked. AddToCart: saffron. BuyNow: navy. Sticky on mobile bottom.
Delivery Info	PIN code input → show estimated delivery date. Free shipping threshold callout.
Product Tabs	Description / Specifications / Reviews — Tailwind tabs; active tab has saffron bottom border.
Review Section	Rating breakdown bar chart. Write review CTA. Paginated review cards.
Related Products	Horizontal scroll strip of ProductCards.

4.4 Cart Page (/cart)

Element	Spec
Cart Items List	Left 60% column. Each item: image thumbnail + name + variant + qty stepper + remove. Price right-aligned.
Order Summary	Right 40% sticky panel. Subtotal, Shipping (free if >₹499), Discount, Total. Coupon input field. Checkout CTA (saffron).
Empty Cart	Centred illustration, 'Your cart is empty', CTA to browse. Suggested products below.
Save for Later	Move item to wishlist with one click.
Price Change Alert	Yellow banner if any item price changed since add: 'Price updated — review before checkout.'

4.5 Checkout (/checkout)

Step	UI Notes
1 – Address	Saved addresses as radio cards. 'Add new' expands inline form (not new page). Google Maps PIN not needed — PIN code autofills city/state.
2 – Shipping	Radio cards: Standard (Free / 5–7 days), Express (₹99 / 2–3 days), Same-day (₹199 / today, PIN dependent).
3 – Payment	Tab switcher: UPI / Card / Net Banking / COD. UPI: QR code + VPA input. Card: inline form with card preview graphic.
Progress Bar	3-step saffron progress indicator at top. Completed steps get checkmark.
Order Summary	Collapsible on mobile. Sticky on desktop right panel.
Security Note	Lock icon + 'Secured by 256-bit SSL' below payment section.

4.6 My Account Pages (/account/*)

Page	Route	Content
Dashboard	/account	Welcome, quick stats (orders, wishlist count), recent orders, recently viewed
Orders	/account/orders	Filterable order list (All/Active/Delivered/Cancelled). Each row: order number, date, items count, total, status badge, View button.
Order Detail	/account/orders/[id]	Status timeline, item list with product images, price breakdown, tracking info, Return/Review CTAs
Profile	/account/profile	Name, email, phone, avatar upload, password change
Addresses	/account/addresses	Address cards grid; Add/Edit/Delete/Set Default
Wishlist	/account/wishlist	Product cards grid with Move to Cart action
Notifications	/account/notifications	List with read/unread state; bulk mark-as-read

4.7 Admin Dashboard (/admin/*)

Section	Route	Key UI
Overview	/admin	4 KPI cards (navy bg, saffron value). Revenue line chart. Orders by status donut. Recent orders table.
Products	/admin/products	Data table with inline status toggle. Bulk select. Add/Edit in slide-over panel.
Orders	/admin/orders	Advanced filter bar. Status update dropdown per row. Bulk status change.

Section	Route	Key UI
Users	/admin/users	Search + role filter. User detail slide-over with ban/unban action.
Categories	/admin/categories	Drag-and-drop tree. Add subcategory inline.
Coupons	/admin/coupons	Create coupon form. Usage stats per coupon.
Reports	/admin/reports	Date-range picker. Revenue, Orders, Inventory tabs. CSV export button.

5. Project Setup & Folder Structure

5.1 Bootstrap Commands

```
npx create-next-app@latest shopforge-web \
  --typescript \
  --tailwind \
  --eslint \
  --app \
  --src-dir \
  --import-alias '@/*'

cd shopforge-web

# Core dependencies
npm install @tanstack/react-query @tanstack/react-query-devtools
npm install zustand immer
npm install axios
npm install react-hook-form @hookform/resolvers zod
npm install next-auth

# UI components
npx shadcn-ui@latest init
npx shadcn-ui@latest add button input label badge card tabs accordion dialog sheet

# Utilities
npm install clsx tailwind-merge class-variance-authority
npm install @radix-ui/react-slot lucide-react
npm install react-hot-toast
npm install react-intersection-observer
npm install date-fns
npm install razorpay

# Dev
npm install -D @tailwindcss/forms @tailwindcss/line-clamp
npm install -D prettier prettier-plugin-tailwindcss
npm install -D @typescript-eslint/eslint-plugin
```

5.2 Folder Structure

```
src/
├── app/
│   ├── (storefront)/
│   │   ├── page.tsx
│   │   ├── c/[slug]/page.tsx
│   │   └── p/[slug]/page.tsx
└── # Next.js App Router
    # Route group – public pages
    # Homepage
    # Category / PLP
    # Product Detail (SSR)
```

```
├── search/page.tsx
├── cart/page.tsx
├── checkout/page.tsx
├── (auth)/ # Route group – auth pages
│   ├── login/page.tsx
│   └── register/page.tsx
├── account/ # Protected – customer
│   ├── layout.tsx
│   ├── page.tsx
│   ├── orders/
│   ├── profile/
│   ├── addresses/
│   └── wishlist/
├── admin/ # Protected – admin
│   ├── layout.tsx
│   ├── page.tsx
│   ├── products/
│   ├── orders/
│   ├── users/
│   ├── categories/
│   ├── coupons/
│   └── reports/
├── api/ # Next.js API routes
│   ├── auth/[...nextauth]/route.ts
│   └── payment/webhook/route.ts
├── layout.tsx # Root layout (fonts, providers)
├── not-found.tsx
└── error.tsx

components/
├── layout/
│   ├── Navbar.tsx
│   ├── Footer.tsx
│   ├── BottomNav.tsx # Mobile only
│   └── AdminSidebar.tsx
├── catalog/
│   ├── ProductCard.tsx
│   ├── ProductGrid.tsx
│   ├── FilterSidebar.tsx
│   ├── SortDropdown.tsx
│   ├── CategoryCard.tsx
│   └── PriceTag.tsx
├── product/
│   ├── ImageGallery.tsx
│   ├── VariantSelector.tsx
│   ├── AddToCartButton.tsx
│   ├── ProductTabs.tsx
│   └── ReviewSection.tsx
├── cart/
│   ├── CartDrawer.tsx
│   ├── CartItem.tsx
│   └── CartSummary.tsx
├── checkout/
│   ├── AddressStep.tsx
│   ├── ShippingStep.tsx
│   └── PaymentStep.tsx
├── search/
│   ├── SearchModal.tsx
│   └── SearchResult.tsx
├── home/
│   ├── HeroBanner.tsx
│   ├── CategoryGrid.tsx
│   ├── FlashSaleSection.tsx
│   └── FeaturedProducts.tsx
└── ui/ # shadcn + custom primitives
    ├── Button.tsx
    └── Badge.tsx
```

```

├── RatingStars.tsx
├── Skeleton.tsx
├── CountdownTimer.tsx
├── lib/
│   ├── api/
│   │   ├── client.ts           # Axios client + typed endpoints
│   │   ├── auth.ts            # Interceptors + refresh
│   │   ├── products.ts
│   │   ├── orders.ts
│   │   └── cart.ts
│   ├── auth.ts                # NextAuth config
│   ├── utils.ts               # cn(), formatPrice(), formatDate()
│   └── constants.ts
├── hooks/
│   ├── useCart.ts
│   ├── useWishlist.ts
│   ├── useSearch.ts
│   ├── useProduct.ts
│   └── useIntersection.ts
├── store/
│   ├── cartStore.ts           # Zustand – cart state
│   ├── uiStore.ts             # Zustand – drawer/modal open states
│   └── wishlistStore.ts
├── types/
│   ├── product.ts
│   ├── order.ts
│   ├── user.ts
│   └── api.ts
├── styles/
│   └── globals.css             # Tailwind directives + CSS vars

```

5.3 Root Layout (app/layout.tsx)

```

import { Plus_Jakarta_Sans, Inter, JetBrains_Mono } from 'next/font/google'
import { Providers } from '@components/Providers'

const jakarta = Plus_Jakarta_Sans({
  subsets: ['latin'], variable: '--font-display',
  weight: ['400', '500', '600', '700', '800']
})
const inter = Inter({
  subsets: ['latin'], variable: '--font-body'
})
const mono = JetBrains_Mono({
  subsets: ['latin'], variable: '--font-mono', weight: ['400']
})

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang='en' className={` ${jakarta.variable} ${inter.variable} ${mono.variable}`}>
      <body className='font-body bg-surface text-navy antialiased'>
        <Providers>
          {children}
        </Providers>
      </body>
    </html>
  )
}

```

5.4 API Client with Token Refresh

```
// lib/api/client.ts
import axios from 'axios'
import { getSession, signOut } from 'next-auth/react'

export const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL,
  withCredentials: true,
})

api.interceptors.request.use(async (config) => {
  const session = await getSession()
  if (session?.accessToken) {
    config.headers.Authorization = `Bearer ${session.accessToken}`
  }
  return config
})

api.interceptors.response.use(
  (res) => res,
  async (error) => {
    if (error.response?.status === 401 && !error.config._retry) {
      error.config._retry = true
      try {
        // NextAuth handles refresh via callbacks
        const session = await getSession()
        if (session?.accessToken) {
          error.config.headers.Authorization = `Bearer ${session.accessToken}`
          return api(error.config)
        }
      } catch {
        signOut({ callbackUrl: '/login' })
      }
    }
    return Promise.reject(error)
  }
)
```

5.5 Cart Store (Zustand)

```
// store/cartStore.ts
import { create } from 'zustand'
import { persist } from 'zustand/middleware'
import { immer } from 'zustand/middleware/immer'

interface CartItem {
  variantId: string; name: string; image: string;
  price: number; quantity: number; sku: string;
}

interface CartStore {
  items: CartItem[]
  isOpen: boolean
  addItem: (item: CartItem) => void
  removeItem: (variantId: string) => void
  updateQty: (variantId: string, qty: number) => void
  clear: () => void
  openCart: () => void
  closeCart: () => void
  total: () => number
  count: () => number
}
```

```
export const useCart = create<CartStore>()(persist(immer((set, get) => ({
  items: [], isOpen: false,
  addItem: (item) => set(state => {
    const existing = state.items.find(i => i.variantId === item.variantId)
    if (existing) existing.quantity += item.quantity
    else state.items.push(item)
    state.isOpen = true
  }),
  removeItem: (id) => set(state => { state.items = state.items.filter(i => i.variantId !== id) }),
  updateQty: (id, qty) => set(state => {
    const item = state.items.find(i => i.variantId === id)
    if (item) item.quantity = Math.max(1, qty)
  }),
  clear: () => set(state => { state.items = [] }),
  openCart: () => set(state => { state.isOpen = true }),
  closeCart: () => set(state => { state.isOpen = false }),
  total: () => get().items.reduce((s, i) => s + i.price * i.quantity, 0),
  count: () => get().items.reduce((s, i) => s + i.quantity, 0),
}))), { name: 'shopforge-cart' })))
```

6. Sprint Development Plan

6 sprints × 2 weeks = 12 weeks. Each sprint ends with a deployable increment to Vercel preview.

Sprint 1 — Foundation + Auth + Layout Shell Week 1–2

Goal: Project boots locally and on Vercel. Auth works. Navbar + footer render.

Task	Details
Next.js setup	Bootstrap, Tailwind config, font loading, folder structure, ESLint/Prettier
Design tokens	tailwind.config.ts with all colors, shadows, animations, font variables
Component primitives	Button (3 variants), Badge, Skeleton, cn() utility, globals.css
NextAuth integration	Credentials provider (email/password) + Google OAuth. JWT strategy. Refresh token rotation in callbacks.
Navbar	Logo, search trigger, cart icon (Zustand count), user menu dropdown, mobile hamburger
Footer	Links, newsletter input, social icons, copyright
BottomNav	Mobile only (md:hidden). Home / Search / Cart / Account tabs. Saffron active state.
CartDrawer	Zustand-driven. Slide-in from right. Empty state. Item list stub.
Cart store	Zustand + persist + immer. All actions + selectors.
Route protection	middleware.ts — redirect /account/* and /admin/* if not authenticated.
Error + loading UI	app/error.tsx, app/loading.tsx, not-found.tsx with on-brand design.
Deploy	Connect GitHub → Vercel. Set NEXT_PUBLIC_API_URL env var.

Sprint 2 — Homepage + Category Listing Week 3–4**Goal: Shoppers can browse homepage and filter a product listing page.**

Task	Details
TanStack Query setup	QueryClientProvider in Providers.tsx. Axios query functions for products + categories.
HeroBanner	SSR via generateMetadata. Rotating product image with CSS animation. CTA links to flash sale.
CategoryGrid	Fetched from /api/v1/categories/tree. 4-col → 2-col. Image cards with overlay label.
FlashSaleSection	CountdownTimer component (real-time). Horizontal scroll product strip. Saffron background.
FeaturedProducts	Static section. ProductCard grid with skeleton loading.
ProductCard	Full spec per Section 3.3. WishlistButton toggle. AddToCartButton (calls Zustand).
PLP — /c/[slug]	generateStaticParams for top categories. Paginated product grid. SSR first page.
FilterSidebar	Accordion: category, brand, price dual-range slider, rating, availability. URL-param driven.
SortDropdown	URL-param driven sort. Persists across pagination.
Active Filters Bar	Removable chips. Clear all. Animated chip entry/exit.
Infinite scroll	react-intersection-observer triggers TanStack Query fetchNextPage.
SEO	generateMetadata per category page. Open Graph image. Structured data (BreadcrumbList).

Sprint 3 — Product Detail + Search Week 5–6**Goal: Shoppers can view products, select variants, and add to cart. Search works.**

Task	Details
PDP — /p/[slug]	Full SSR. generateStaticParams for top products. ISR with revalidate: 60.
ImageGallery	Thumbnail strip + main image. Lightbox modal on desktop click. Touch swipe on mobile.
VariantSelector	Color swatches + size pills. OOS variants: strikethrough + disabled. URL query param sync.
PriceTag	Sale price in saffron. Original strikethrough slate. Savings badge (e.g. '18% off').
AddToCartButton	Optimistic update. Saffron pulse animation on cart icon. Loading spinner during API call.
BuyNow	Add to cart then immediately route to /checkout.
Delivery	PIN code input → debounced API call → delivery date display.

Task	Details
estimator	
ProductTabs	Description (HTML render) / Specs (key-value table) / Reviews. Active tab: saffron underline.
ReviewSection	Rating breakdown (bar chart with CSS). Write review form (modal). Paginated review cards.
SearchModal	cmd+K / search icon trigger. Debounced 300ms. Autocomplete suggestions. Search history.
Search page /search	Full results page. Reuses ProductGrid + FilterSidebar. Handles ?q= param.
Related products	Horizontal scroll strip. Fetched from /api/v1/products/{id}/related.
Breadcrumb	Schema.org BreadcrumbList. Saffron separator character.
Wishlist toggle	Heart button on card + PDP. Zustand wishlist store. Sync to API if authenticated.

Sprint 4 — Cart + Checkout + Payment Week 7–8

Goal: Full purchase flow end-to-end. Razorpay integrated. Order confirmation page.

Task	Details
Cart page /cart	CartItem with qty stepper. Remove item. Save for later. Price change alert banner.
CartSummary	Subtotal, shipping, coupon discount, total. Coupon input with validate API call.
Checkout layout	3-step progress bar. Collapsible order summary sidebar. Step state in Zustand/URL.
AddressStep	Saved address radio cards. 'Add new' inline form (react-hook-form + zod). PIN autofill.
ShippingStep	Radio cards with price + ETA. Conditional same-day based on PIN code.
PaymentStep	Tab: UPI (QR + VPA) / Card (card preview widget) / Net Banking (bank select) / COD.
Razorpay integration	Load Razorpay script on checkout mount. Create order via backend. Open checkout modal. Verify signature.
Order success /orders/[id]/success	Confetti animation. Order number in mono font. Estimated delivery. CTA to track / continue shopping.
Guest checkout	Email gate. Guest session cookie. Merge cart if user later registers.
Form validation	All checkout forms: react-hook-form + zod schemas. Inline field errors in danger red.
Loading states	Full-page spinner overlay during payment processing. Disable all inputs.
Error handling	Payment failure page with retry CTA. API error toasts via react-hot-toast.

Sprint 5 — Account Pages + Order Management Week 9–10**Goal: Full account section. Order tracking. Return flow. Wishlist.**

Task	Details
Account layout	Left nav sidebar (desktop) / tab bar (mobile). Active item: saffron left border.
Account dashboard	Welcome section, KPI mini-cards, recent orders table, recently viewed products.
Orders list	Filter tabs: All / Active / Delivered / Cancelled. Sortable table. Status badges.
Order detail	Status timeline (CSS vertical stepper, completed steps in saffron). Item list. Invoice download.
Return flow	Request return button → modal: select items, reason, upload image. Status tracking.
Profile page	Avatar upload (crop + preview). Form with name, email, phone. Change password section.
Addresses page	Address cards grid. Add/Edit: slide-over panel. Set default: radio-style selection.
Wishlist page	ProductCard grid. 'Move to cart' action. Remove from wishlist.
Notifications page	List view. Unread: white bg. Read: slate/5 bg. Bulk mark-read button.
Admin guard	Separate layout.tsx for /admin. Role check: redirect CUSTOMER role to /account.

Sprint 6 — Admin Dashboard + Polish + Perf + Deploy Week 11–12**Goal: Admin fully operational. Performance targets hit. Production deployed.**

Task	Details
Admin overview	KPI cards (recharts LineChart for revenue). Orders donut chart. Recent orders table.
Admin products	TanStack Table with search, sort, filter. Inline status toggle. Add/Edit in Sheet panel.
Admin orders	Advanced filter bar. Row status dropdown. Bulk update selected.
Admin users	User table. Detail slide-over. Ban/unban with confirmation dialog.
Admin categories	Drag-and-drop tree (dnd-kit). Inline add subcategory.
Admin coupons	Create form + coupon table with usage stats bar.
Admin reports	Date range picker. Revenue/Orders/Inventory tabs. Recharts. CSV export.
Performance audit	Lighthouse ≥ 90 on PDP. next/image on all product images. Font display swap.
SSR / ISR strategy	PDP: ISR revalidate 60s. PLP: SSR. Homepage: SSG + client hydration for dynamic sections.
SEO final pass	sitemap.xml generation. robots.txt. Structured data: Product, BreadcrumbList, WebSite.

Task	Details
Accessibility	All interactive elements keyboard navigable. Focus rings (saffron shadow). ARIA labels on icon buttons.
Error monitoring	Sentry integration. Track client + server errors.
Production deploy	Vercel prod domain. Edge middleware for auth. Environment variables audit.

7. Performance & SEO Targets

Metric	Target	Strategy
LCP (Largest Content Paint)	< 2.5s	next/image with priority on hero. Preload hero image. CDN.
FID / INP	< 100ms	No heavy JS on main thread. Defer non-critical scripts.
CLS (Layout Shift)	< 0.1	Explicit image dimensions. Skeleton placeholders.
Bundle size (JS)	< 180KB initial	Dynamic imports for checkout, admin. Tree-shake icon libs.
Lighthouse Score	> 90 all categories	Run in CI with lighthouse-ci. Block merge if regression.
Core SEO	Indexed product pages	SSR/ISR for PDP. generateMetadata per page. Sitemap.xml.
Mobile PageSpeed	> 85	Bottom nav. No horizontal scroll. Touch targets > 44px.

8. Environment Variables

```
# .env.local
NEXT_PUBLIC_API_URL=http://localhost:8080
NEXT_PUBLIC_APP_URL=http://localhost:3000

# NextAuth
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=<random-32-char-string>

# OAuth
GOOGLE_CLIENT_ID=<from-google-cloud-console>
GOOGLE_CLIENT_SECRET=<from-google-cloud-console>

# Razorpay
NEXT_PUBLIC_RAZORPAY_KEY_ID=rzp_test_xxxx
RAZORPAY_KEY_SECRET=<server-only-never-public>

# Sentry (optional)
NEXT_PUBLIC_SENTRY_DSN=https://xxx@sentry.io/xxx
```

9. Senior Tips — Next.js Specific

Pitfall	Wrong	Correct
Server vs Client components	'use client' on everything	Default to Server Component. Only add 'use client' for hooks, events, browser APIs.
Data fetching	useEffect + fetch in all components	fetch() in Server Components. TanStack Query only in Client Components that need real-time updates.
Image optimisation		Always <Image> from next/image. Set sizes prop for responsive.
Metadata	Static <title> in layout	generateMetadata() per page — async, can fetch from DB/API.
Route params	useRouter() in Server Component	Server Components receive params as props — no router hook needed.
Auth in middleware	Fetch user on every render	middleware.ts JWT check — redirect before page renders.
Fonts	Google Fonts <link> tag	next/font/google — self-hosted, zero layout shift, automatic subset.
Loading UI	Custom loading states everywhere	app/loading.tsx auto-wraps in Suspense — use it per route segment.
Error handling	try/catch in every component	error.tsx per segment. 'use client' required. Access error + reset().
ISR vs SSG	getStaticProps (Pages Router)	App Router: fetch() + revalidate option in Server Components.